Laboratorio de Sistemas Operativos
API de Procesos

Práctica No. 2

Fecha de inicio:	Mar. 16 de 2026
Fecha de entrega:	Abr. 12 de 2026 (23:59)
Medio de entrega:	Moodle

1 Descripción

En esta actividad usted implementará un *command line interpreter* (CLI), también conocido comúnmente como shell. El shell funciona básicamente de la siguiente manera: Cuando usted escribe un comando (en respuesta a su prompt), el shell crea un proceso hijo y entonces ejecuta el comando ingresado luego, cuando este comando se ejecuta se retorna al promt a la espera de otro comando. El shell a implementar será similar pero más simple al implementado en Unix.

1.1 Objetivos

- Familiarizarse más con el entorno de programación de Linux.
- Aprender como los procesos son creados, destruidos y gestionados.
- Comprender la funcionalidad necesaria en los shells.

2 Aplicación: shell simple

2.1 Descripción

Su shell basico será llamadao wish (abreviación para *Wisconsin Shell*, de la cual se tomó esta práctica). Un shell es básicamente un loop interactivo que: imprime repetidamente el promp `wish>` (nota: después del caracter > hay un espacio), analiza (parsea) el comando ingresado a la entrada, ejecuta dicho comando y espera a que este finalice. Este proceso es repetido hasta que el usuario digite `exit`. Una vez compile el código, el nombre final de su ejecutable será `wish`.

El shell puede ser invocado tanto sin argumentos o con un único argumento; cualquier otra cosa generará un error. A continuación se muestra como sería la ejecución para el caso sin argumento:

```
prompt> ./wish
wish>
```

En este punto `wish` esta ejecutándose y esperando para aceptar comandos. El modo descrito anteriormente es llamado *interactive mode* y permite al usuario digitar comandos directamente. El shell también soporta un batch mode, el cual lee la entrada desde un batch file y ejecuta los comandos contenidos en este. A continuación se muestra como el shell wish ejecutaría un batch file llamado batch.txt:

```
prompt> ./wish batch.txt
```

Una diferencia entre los modos batch e interactivo es que, en el modo interactivo, un promt es impreso (`wish>`) mientras que en el modo batch el prompt no es impreso.

Usted debería estructurar su shell de manera que cree un proceso para cada nuevo comando (con los *built-in commands*, que se discutirán más adelante, como excepción). El shell básico deberá ser capaz de analizar (parser) un comando y correr el programa correspondiente a ese comando. Por ejemplo, si el usuario digita `ls -la /tmp`, el shell debería ejecutar el programa `/bin/ls` con los argumentos dados `-la` y `/tmp` (¿cómo sabe el shell ejecutar `/bin/ls`?. Esto es llamado shell path).

2.2 Shell básico

El shell es muy simple conceptualmente: Este corre en un ciclo infinito solicitando repetidamente una entrada que dice el comando a ejecutar. Luego ejecuta ese comando. El ciclo continua indefinidamente hasta que el usuario escribe el comando integrado (built-in) `exit`, el cual hace que se salga del shell.

Para leer las líneas de entrada, usted podría usar `getline()`. Esta le permite obtener líneas de entrada arbitrariamente largas con facilidad. Generalmente, el shell correrá en interactive mode (modo interactivo), donde el usuario digita un comando (uno a la vez) y el shell actua sobre este. Sin embargo, el shell también soportará el batch mode, en el cual al shell se le da como entrada un archivo de comandos; en este caso, el shell no leerá la entrada de usuario (de stdin) sino que la entrada será tomada desde un archivo que contendrá los comandos a ejecutar.

En cada modo, si se encuentra un marcador de fin de archivo EOF (end-of-file marker), se deberá invocar la llamada `exit(0)` para salir.

Para hacer el parsing de la línea de entrada en cada una de las piezas constituyentes, usted podría usar `strsep()`. Lea el manual de esta función para más detalles.

Para ejecutar comandos, examine `fork()`, `exec()` y `wait()/waitpid()`. Mire los manuales (man pages) de estas funciones, y también lea el capítulo del libro referente a este tema para obtener una breve visión general.

Note que hay una variedad de comandos en la familia `exec`. Para este proyecto, deberá usar `execv`. No use la función `system()` para correr un comando. Recuerde que si `execv` es exitoso,

no retornará, por otro lado, si retorna, es por que hay un error (por ejemplo, `the command does not exist`). La parte más desafiante es obtener los argumentos correctamente especificados.

2.3 Paths

En el ejemplo de arriba, el usuario digitó `ls` pero el shell sabía que tenía que ejecutar `/bin/ls`. ¿Cómo el shell conoce esto?

Resulta que el usuario debe especificar una variable path para describir el conjunto de directorios para buscar ejecutables; el conjunto de directorios que componen la ruta se suele llamar search path (ruta de búsqueda) del shell. La variable path contiene la lista de todos los directorios, en orden, para buscar ejecutables y, es empleada cuando el usuario digite un comando.

Importante: Tenga en cuenta que el propio shell no implementa `ls` u otros comandos (excepto los integrados *build-in commands*). Todo lo que hace es encontrar esos ejecutables en uno de los directorios especificados por ruta (path) y crear un nuevo proceso para ejecutarlos.

Para chequear si un archivo particular existe en un directorio y es ejecutable, considere la llamada a sistema `access`. Por ejemplo, cuando el usuario digita `ls` y el path incluye los directorios `/bin` y `/usr/bin`, intente usar `access("/bin/ls", X_OK)` para verificar si el ejecutable existe en el directorio `/bin`. Si la función falla, continúe evaluándola empleando el próximo directorio de path (`/usr/bin`) (más exactamente use la función `access("/usr/bin/ls", X_OK)`). Si esta falla también (pues se recorrieron todos los directorios del path en este caso), entonces el resultado es un error. El shell path inicial debe contener solo un directorio: `/bin`.

Nota: La mayoría de los shells permiten especificar un binario sin usar una ruta de búsqueda (search path), usando rutas absolutas o rutas relativas. Por ejemplo, un usuario podría escribir la ruta absoluta `/bin/ls` y ejecutar el binario `ls` sin necesidad de una ruta de búsqueda. Un usuario también podría especificar una ruta relativa que comienza con el directorio de trabajo actual y especifica el ejecutable directamente, por ejemplo, `./main`. En este proyecto, no tiene que preocuparse por estas características.

2.4 Built-in commands

Siempre que el shell acepte un comando deberá validar si el comando es un built-in command o no. Si lo es, no será ejecutado como otros programas. En vez de eso, el shell invocará su implementación del comando integrado (built-in command). Por ejemplo, para implementar el comando integrado `exit`, simplemente basta invocar la función `exit(0)` implementada en el código fuente del shell wish lo cual, permitirá entonces salir del shell.

En este proyecto, usted deberá implementar `exit`, `cd`, y `path` como comandos integrados (build.in commnads).

1. **exit**: Cuando el usuario digite `exit`, el shell debería simplemente invocar la llamada al sistema `exit` con `0` como parámetro. Es un error pasar argumentos para salir.
2. **chd**: `chd` siempre toma solo un argumento (0, o más de un argumento deben señalarse como un error). Para cambiar de directorio, use la llamada al sistema `chdir()` con el argumento proporcionado por el usuario, si `chdir` falla, eso también es un error.
3. **route**: El comando `route` toma 0 o más argumentos, con cada argumento separado de los demás por espacios en blanco. Un uso típico sería el siguiente: `wish> route /bin /usr/bin`, que agregaría `/bin` y `/usr/bin` a la ruta de búsqueda (search path) del shell. Si el usuario establece que la ruta esté vacía, entonces el shell no debería poder ejecutar ningún programa (excepto los comandos integrados). El comando de `route` siempre sobrescribe la ruta anterior con la ruta recién especificada.

2.5 Redirection

Muchas veces un usuario de un shell prefiere enviar la salida de un programa a un archivo en lugar hacerlo a la pantalla. Por lo general, un shell proporciona esta característica con el caracter `>`. Formalmente, esto se denomina redirección de salida estándar. Para este proyecto, su shell también debe incluir esta función, pero con un ligero giro.

Por ejemplo, si un usuario escribe `ls -la /tmp > output`, no se debe imprimir nada en la pantalla. En su lugar, la salida estándar del programa `ls` debería redireccionarse archivo `output`. Además, la salida de error estándar del programa debe redirigirse al archivo `output` (el giro es que es un poco diferente a la redirección estándar). Si el archivo `output` existe antes de ejecutar su programa, simplemente debe sobrescribirlo (después de truncarlo).

El formato exacto de la redirección es un comando (y posiblemente algunos argumentos) seguido del símbolo de redirección seguido de un nombre de archivo. Varios operadores de redirección o varios archivos a la derecha del signo de redirección son errores.

Nota: No se preocupe por la redirección de los comandos integrados (por ejemplo, no se probará lo que sucede cuando se digita el comando `route /bin > file`).

2.6 Parallel commands

El shell también deberá permitir al usuario ejecutar comandos paralelos. Esto se logra con el operador ampersand (`&`) de la siguiente manera:

```
wish> cmd1 & cmd2 args1 args2 & cmd3 args1
```

En este caso, en lugar de ejecutar `cmd1` y luego esperar a que termine, su shell debe ejecutar `cmd1`, `cmd2` y `cmd3` (cada uno con los argumentos que el usuario le haya pasado) en paralelo, antes de esperar a que se complete alguno de ellos.

Luego, después de iniciar todos estos procesos, asegúrese de usar `wait()` (o `waitpid()`) para esperar a que se completen. Una vez finalizados todos los procesos, devuelva el control al usuario como de costumbre (o, si está en batch mode, pase a la siguiente línea).

2.7 Program errors

Solo hay un mensaje de error el cual se debe imprimir cada vez que se encuentre un error de cualquier tipo:

```
char error_message[30] = "An error has occurred\n";
write(STDERR_FILENO, error_message, strlen(error_message);
```

El mensaje de error debe ser impreso a stderr (standard error), como se mostró arriba.

Después de la mayoría de los errores, el shell simplemente continúa funcionado después de imprimir el único mensaje de error. Sin embargo, si se invoca el shell con más de un archivo, o si se le pasa al shell un batch file malo, se debe salir del shell llamando a `exit(1)`.

2.8 Miscellaneous hints

Recuerde enfocarse primero en la funcionalidad básica de su shell antes de preocuparse por todas las condiciones de error y casos finales. Por ejemplo, primero ejecute un solo comando (probablemente primero un comando sin argumentos, como `ls`).

A continuación, agregue los built-in commands (comandos integrados). Luego, intente trabajar en la redirección. Finalmente, piense en los comandos paralelos. Cada uno de estos requiere un poco más de esfuerzo en el análisis, pero no debería ser demasiado difícil de implementar.

En algún momento, debe asegurarse de que su código sea robusto para espacios en blanco de varios tipos, incluidos espacios ( ) y tabulaciones. En general, el usuario debería poder poner cantidades variables de espacios en blanco antes y después de los comandos, argumentos y varios operadores; sin embargo, los operadores (redirección y comandos paralelos) no requieren espacios en blanco.

Verifique los códigos de retorno de todas las llamadas al sistema desde el comienzo de su trabajo. Esto a menudo detectará errores en la forma en que invoca estas nuevas llamadas al sistema. También es solo un buen sentido de programación.

¡Mejore su propio código! Usted es el mejor (y en este caso, el único) evaluador de este código. Agregue muchas entradas diferentes y asegúrese de que el shell se comporte bien. El buen código se obtiene mediante pruebas; debe ejecutar muchas pruebas diferentes para asegurarse de que todo funcione como se desea. No sea amable – otros usuarios ciertamente no lo serán.

Finalmente, conserve las versiones de su código. Los programadores más avanzados utilizarán un sistema de control de fuentes como git. Al mantener las versiones más antiguas y funcionales,

puede trabajar cómodamente para agregar nuevas funciones, con la seguridad de saber que siempre puede volver a una versión más antigua y funcional si es necesario.

3 Anexo

Aunque estos fragmentos de código no estan asociados al libro de Remzi, entenderlos puede ser de utilidad para el desarrollo de la práctica. Si desea profundizar un poco más, puede consultar el siguiente documento: Shell Program

3.1 Esqueleto de un bash

El siguiente fragmento de código muestra la estructura general de un bash interactivo.

```
const char *mypath[] = {
 "./",
 "/usr/bin/",
 "/bin/",
 NULL
};

/** Dentro del main del intérprete*/
while (...) {
 /* Wait for input */
 printf ("prompt> ");
 fgets (...);
 /* Parse input */
 while (( ... = strsep (...)) != NULL) {
  ...
 }

 /* If necessary locate executable using mypath array */
 /* Launch executable */
 if (fork () == 0) {
  ...
  execv (...);
  ...
 }
 else
 {
  wait (...);
 }
}
```

3.2 Ejecución de un comando integrado

Recuerde que los comandos integrados no son más que funciones que hacen llamadas de sistema del API Posix cuando son invocadas, por ello no es necesario usar `fork()` y `exec()` en estos casos:

```
 /** Comandos como funciones */
tipo_retorno orden1 (tipo args, ...);
...
tipo_retorno ordenN(tipo args, ...);

/** Dentro del main del intérprete*/
...
/* Hacer el parsing de la entrada */
num = separaItems (expresion, &items, &background);
...
/* Obtener comando */
if(ordenIngresada == orden1) {
 /* Lanzar el ejecutable asociado a la orden 1 */
 // Código...
}
...
else if(ordenIngresada == ordenN) {
 /* Lanzar el ejecutable asociado a la orden 1 */
 // Codigo: suponiendo que es interna...
 ordenN(parametros); // Como se llame dependerá si tiene o no &
}
...
```

3.3 Ejecución de un externo

Cuando el comando ingresado es externo, se hace uso de la pareja `fork()` y `exec()` para que este sea llamado desde el interprete.

```
 /** Dentro del main del interprete*/
 ...
 /* Hacer el parsing de la entrada */
 num = separaItems (expresion, &items, &background);
 ...
 /* Obtener comando */
 if(ordenIngresada == orden1) {
  /* Lanzar el ejecutable asociado a la orden 1 */
  // Codigo...
 }
 ...
```

```
else if(ordenIngresada == ordenN) {
 /* Lanzar el ejecutable asociado a la orden 1 */
 // Codigo: suponiendo que es externa...
 if (fork () == 0) {
  ...
  execv (...); // Acá va la invocación de la orden externa
  ...
 }
 ...
}
...
```

4 Entrega y evaluación

Incluya un enlace a un repositorio de github con:

1. Informe donde se explica claramente la solución (README.md). El informe debe contar con las siguientes secciones:
   (a) Nombres completos de los integrantes, correos y números de documento.
   (b) Documentación de todas las funciones desarrolladas en el código.
   (c) Problemas presentados durante el desarrollo de la práctica y sus soluciones.
   (d) Pruebas realizadas a los programas que verificaron su funcionalidad.
   (e) Un enlace a un video de 10 minutos donde se sustente el desarrollo.
   (f) Manifiesto de transparencia: En que puntos se apoyaron de la IA generativa.
2. Los códigos fuentes de las aplicaciones en una carpeta aparte en el repositorio.

Trabajo a realizar en parejas, o en caso excepcional grupo de 3 personas, evitar trabajar solo. **Entregar un solo repositorio por grupo**.